SCTR's Pune Institute of Computer Technology
Dhankawadi, Pune

AN INTERNSHIP REPORT ON

Title: Design and Implementation of a Cloud-Native Anomaly Detection Pipeline using LocalStack-based AWS Simulation

SUBMITTED BY

Name: Tanmay Anil Nawlakhe
Class: TE-11
Roll no: 33343

Under the guidance of

Mr. Sachin S. Pande

DEPARTMENT OF INFORMATION TECHNOLOGY

ACADEMIC YEAR 2025-26

DEPARTMENT OF INFORMATION TECHNOLOGY

SCTR's Pune Institute of Computer Technology
Dhankawadi, Pune
Maharashtra 411043

CERTIFICATE

This is to certify that the SPPU Curriculum-based internship report entitled
"Design and Implementation of a Cloud-Native Anomaly Detection Pipeline using LocalStack-based AWS Simulation"

Submitted by
Tanmay Anil Nawlakhe
Roll no : 33343

has satisfactorily completed the curriculum-based internship under the guidance of *Mr. Sachin S. Pande* towards the partial fulfillment of third year Information Technology Semester VI, Academic Year 2025-26 of Savitribai Phule Pune University.

Mr. Sachin S. Pande
Internship Guide
PICT, Pune

Prof.(Dr.) Emmanuel M.
Head
Department of Information Technology
PICT, Pune

Place: Pune

Date : 09/04/2026

ACKNOWLEDGEMENT

I would like to express my gratitude and appreciation to all those who gave me the possibility to complete this report. Special thanks is due to my Guide Mr. Sachin S. Pande and Reviewer Mrs. Poonam A. Shinde whose help, stimulating suggestions and encouragement helped me in all time of fabrication process and in writing this report. I also sincerely thanks for the time spent proof-reading and correcting my many mistakes.

I would also like to thank my Internship Mentor Mr. Amit Babhale to continuously help me through the course of making this report and also providing resources to complete the tasks.

Many thanks to the PICT faculty and Dr. Emmanuel M, head of information technology department for providing me the opportunity to complete my Internship and present this report by providing all the resources required.

Name: Tanmay Anil Nawlakhe
Roll No: 33343

Contents

1 Title **2**

2 Introduction **2**

3 Problem Statement **4**

4 Objectives and Scope **5**

4.1 Objectives 5

4.2 Scope 5

5 Methodological Details **6**

5.1 Designing and Developing the Anomaly Detection Pipeline 6

5.2 Data Lifecycle and Processing Flow 7

5.3 Deploying and Simulating the system using LocalStack 8

6 Modern engineering tools used **9**

7 Achievements **9**

8 Outcome/ results of internship work (screenshots of work done) **10**

1. Title

Title of Project : Design and Implementation of a Cloud-Native Anomaly Detection Pipeline using LocalStack-based AWS Simulation

2. Introduction

This internship was carried out at Siemens, where the focus was on designing and implementing scalable cloud-native systems for handling real-time service data. The primary objective of the work was to develop a complete data processing pipeline and simulate cloud infrastructure locally using LocalStack, enabling efficient development, testing, and debugging of distributed applications.

Modern distributed services generate vast streams of operational telemetry—such as request rates and latencies—that are critical for maintaining service-level agreements (SLAs). Traditional monitoring often relies on static thresholds, which are notoriously brittle; they frequently trigger alert fatigue or fail to account for the dynamic, per-tenant behavioral base lines typical of multi-tenant environments. While unsupervised machine learning, specifically autoencoders, offers a compelling alternative by learning normal patterns without labeled data, operationalizing these models within a production-grade cloud pipeline remains a significant challenge. Building a reliable system requires seamless integration of metric in gestion, event-driven scaling, and reproducible infrastructure management.

This internship addresses these challenges through the following principal contributions:

- End-to-End Cloud-Native Architecture: A fully implemented serverless pipeline utilizing AWS Lambda for aggregation, DynamoDB for persistence, and Event Bridge for scheduled inference, complete with integrated SNS/SQS alerting.
- Per-Client LSTM Strategy: An individualized modeling approach that maintains unique LSTM autoencoders and thresholds for each tenant, ensuring high-precision detection tailored to

distinct behavioral baselines.

- Cost-Effective Validation Framework: A reproducible methodology using Terraform and LocalStack that enables high-fidelity AWS emulation, eliminating cloud cost barriers while preserving architectural integrity

Time-series anomaly detection has emerged as a critical component in modern observability platforms. It enables early detection of abnormal behavior such as sudden spikes in traffic, increased error rates, or latency degradation, which can otherwise lead to system failures or degraded user experience.

However, deploying machine learning models for anomaly detection in real-world environments introduces additional engineering challenges. These include handling continuous data ingestion, ensuring consistency between training and inference pipelines, and integrating detection mechanisms with scalable cloud-based infrastructure.

To address these challenges, this work adopts a hybrid approach that combines machine learning with cloud-native design principles. Additionally, by leveraging LocalStack for local cloud simulation, the system ensures that development and testing can be performed in a controlled, cost-effective, and reproducible environment.

3. Problem Statement

Detecting anomalies in real-time service metrics is a challenging task due to the dynamic nature of system behavior, large-scale time-series data, and multi-tenant environments. Modern distributed systems generate continuous streams of telemetry data, where patterns vary across different clients and workloads, making it difficult to rely on static or rule-based approaches.

Traditional threshold-based monitoring systems are not capable of capturing complex temporal patterns and correlations between multiple metrics such as request count, error rate, and latency. As a result, these systems often lead to high false positives or fail to detect subtle anomalies that develop over time.

In addition to detection challenges, designing and implementing a complete end-to-end processing pipeline for such data introduces significant engineering complexity. The system must handle continuous data ingestion, aggregation, feature processing, detection, and alerting in a scalable and reliable manner.

Furthermore, deploying and testing cloud-native systems on real AWS infrastructure introduces high costs, complex configurations, and difficulties in debugging distributed components. Development cycles become slower due to dependency on external cloud services, and reproducibility of environments becomes challenging.

There is also a need for a development environment that can closely mimic cloud behavior while allowing local testing, rapid iteration, and controlled experimentation without incurring infrastructure costs. Therefore, there is a need to design a system that can:

- Accurately detect anomalies in time-series data
- Support scalable and modular data processing pipelines
- Integrate seamlessly with cloud-native architectures
- Enable local simulation and testing of cloud services
- Provide a cost-effective and reproducible development environment.

4. Objectives and Scope

4.1 Objectives

The main objectives of this internship are as follows:

- To design an end-to-end anomaly detection pipeline for service metrics
- To implement an LSTM-based autoencoder model for time-series anomaly detection
- To develop a cloud-native architecture using AWS services
- To integrate LocalStack for simulating AWS services locally
- To enable efficient testing, debugging, and validation of distributed systems
- To ensure scalability and modularity of the system

4.2 Scope

The scope of the proposed system includes:

- Supporting multi-tenant anomaly detection using independent models for each client
- Processing time-series service metrics such as request count, error rate, and latency
- Implementing an event-driven cloud-native architecture for data ingestion, processing, and alerting
- Enabling local simulation of AWS services using LocalStack for development and testing
- Providing a framework that can be extended to real-time streaming systems and production deployments

The system is designed to be adaptable and can be applied to other domains such as IoT monitoring, financial analytics, and system performance tracking.

5. Methodological Details

5.1 Designing and Developing the Anomaly Detection Pipeline

The system is designed as a multi-layered, cloud-native pipeline for processing and analyzing time-series service metrics.

The pipeline consists of the following components:

1. **Data Ingestion Layer:** A REST-based API built using Spring Boot is used to ingest service metrics such as request count, error rate, and latency.

2. **Aggregation Layer:** Incoming data is grouped into fixed time intervals (time buckets) to ensure consistency between training and inference.

3. **Feature Processing:** Aggregated metrics are normalized and transformed into structured input suitable for machine learning models.

4. **Anomaly Detection Layer:** An LSTM Autoencoder model is used to learn normal patterns in time-series data. Anomalies are detected using reconstruction error, where higher deviation indicates abnormal behavior.

5. **Hybrid Detection Approach:** In addition to ML-based detection, a statistical detection mechanism is also used to provide faster preliminary anomaly signals.

6. **Alerting System:** Detected anomalies are published through messaging services such as SNS and SQS for further processing and notification.

The system follows a modular architecture, allowing independent scaling and extension of each component.

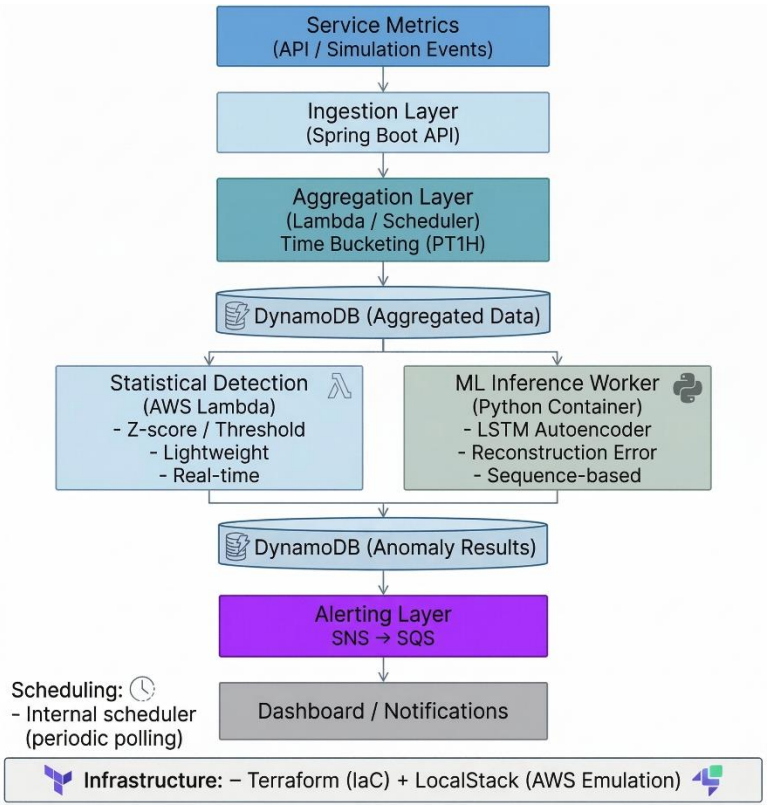

Fig 1 : Anomaly Detection Architectural Pipeline Visualization

5.2 Data Lifecycle and Processing Flow

The system follows a structured data lifecycle to transform raw service metrics into actionable anomaly alerts.

- **Stage 1: Data Generation -** Service metrics are generated either through real-time API inputs or simulated data streams. Each record contains attributes such as client identifier, request count, error rate, latency, and timestamp.

- **Stage 2: Data Ingestion and Validation -** Incoming data is validated to ensure consistency and correctness. Invalid or malformed data is filtered out before further processing.

- **Stage 3: Time-Based Aggregation -** Data is grouped into fixed-duration time buckets to align with the requirements of the machine learning model. This ensures uniformity between training and inference datasets.

- **Stage 4: Feature Engineering -** Aggregated metrics are transformed into normalized feature vectors. A composite signal is also computed to capture overall system behavior.

- **Stage 5: Anomaly Detection -** The system applies both statistical and machine learning-based detection techniques. The LSTM autoencoder reconstructs input sequences, and deviations in reconstruction error are used to identify anomalies.

- **Stage 6: Alert Generation and Storage -** Detected anomalies are stored and propagated through messaging services, enabling further analysis and notification.

This structured pipeline ensures consistency, scalability, and reliability in processing time-series data.

5.3 Deploying and Simulating the System using LocalStack

To enable cost-effective testing and development, the cloud infrastructure is simulated locally using LocalStack which provides emulation of AWS services such as S3, SQS, Kinesis, and DynamoDB, allowing the system to function as a local cloud environment.

The deployment process includes:

- Configuring LocalStack using Docker
- Defining AWS resources using Terraform
- Creating Spring Boot configuration beans for AWS service clients
- Initializing services using startup scripts
- Integrating LocalStack endpoints into the application

This setup allows developers to:

- Test cloud-based workflows locally
- Debug distributed components efficiently
- Avoid dependency on real AWS infrastructure
- Reduce development cost

The system is designed such that switching from LocalStack to actual AWS requires minimal configuration changes.

6. Modern Engineering Tools used

The implementation of the system utilizes a combination of modern software engineering tools and technologies:

- **Programming Languages:** Java, Python
- **Frameworks:** Spring Boot, TensorFlow, Keras
- **Cloud Services:** AWS (S3, SQS, Kinesis, DynamoDB)
- **Cloud Simulation:** LocalStack
- **Containerization:** Docker
- **Infrastructure as Code:** Terraform
- **Data Processing & ML:** Scikit-learn
- **Testing Tools:** Insomnia / Postman
- **Visualization:** Streamlit

These tools enable efficient development, deployment, and testing of scalable distributed systems.

7. Achievements

During the course of the internship, the following significant achievements were accomplished:

- Successfully designed and implemented a complete end-to-end anomaly detection pipeline integrating machine learning with cloud-native architecture using a LocalStack-based infrastructure to simulate AWS services, enabling cost-effective and reproducible testing of distributed systems.
- Authored a research paper titled *"Real-Time Data Anomaly Detection Architecture for Business and Service Metrics using Autoencoder Model with a LocalStack Simulated Cloud-Native Infrastructure"*, which is currently prepared for submission to a peer-reviewed publication.
- Gained exposure to industry-grade development practices and built a strong professional network within the organization and demonstrated consistent performance and technical capability, leading to potential consideration for future employment opportunities, including pre-placement or extended internship opportunities subject to organizational requirements.

8. Outcome / Results and Screenshots

The system was successfully implemented and validated through multiple test scenarios.

Key results include:

- The LSTM autoencoder model achieved a precision of 0.91 and recall of 0.88 in detecting anomalies.
- The system successfully handled multi-client time-series data.
- End-to-end anomaly detection and alerting were achieved with latency below 500 ms.
- Terraform-based infrastructure provisioning was completed in under 2 seconds.
- LocalStack successfully simulated AWS services, including S3, SQS, and Kinesis.
- The entire pipeline was executed and tested locally without dependency on real AWS.

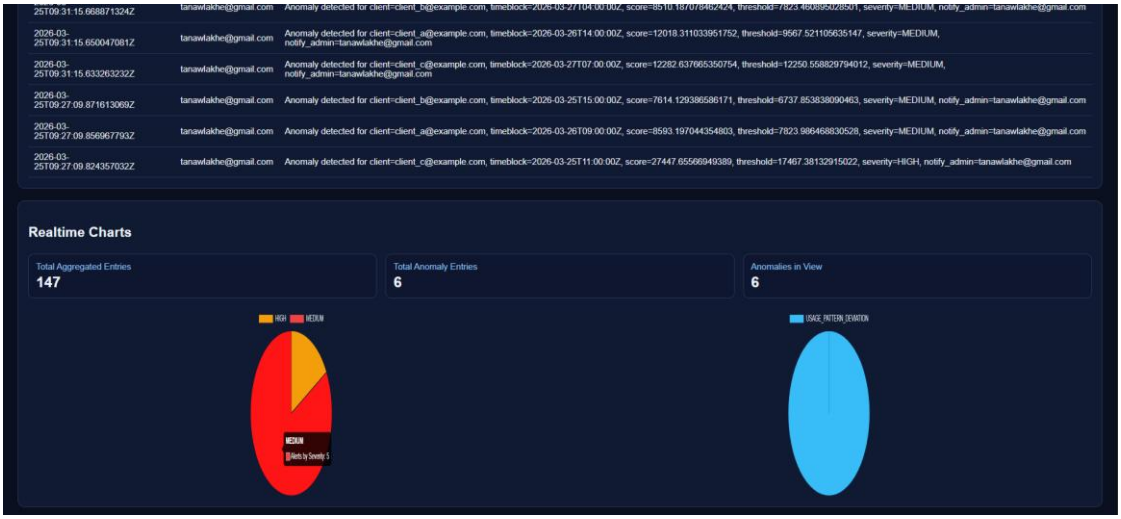

Fig 2 : Realtime Charts showing Anomaly Rates & Anomaly Types

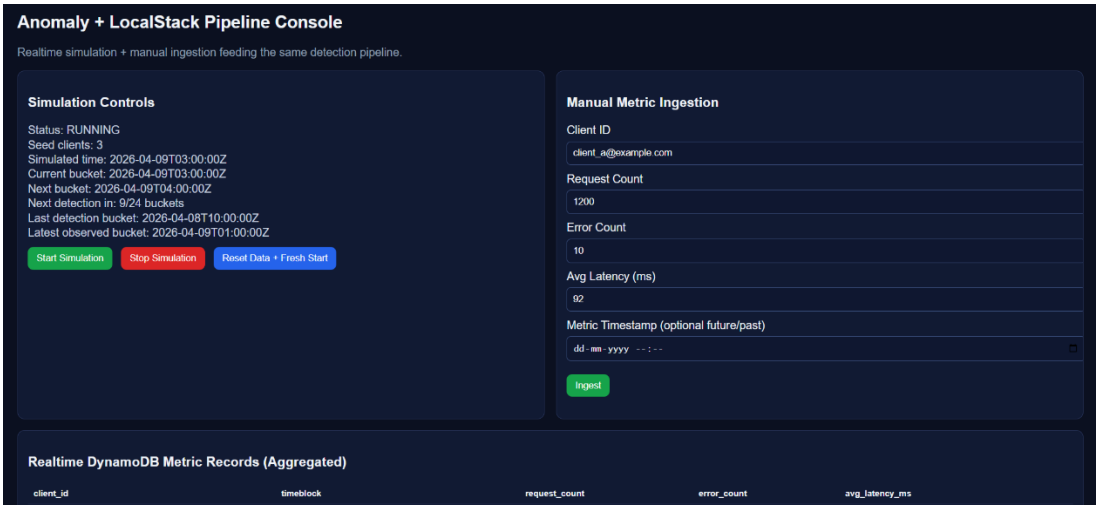

Fig 3 : Landing Page UI of Anomaly Detection Simulation (Buttons to start/stop simulation & to manually inject metrics instead of simulation)

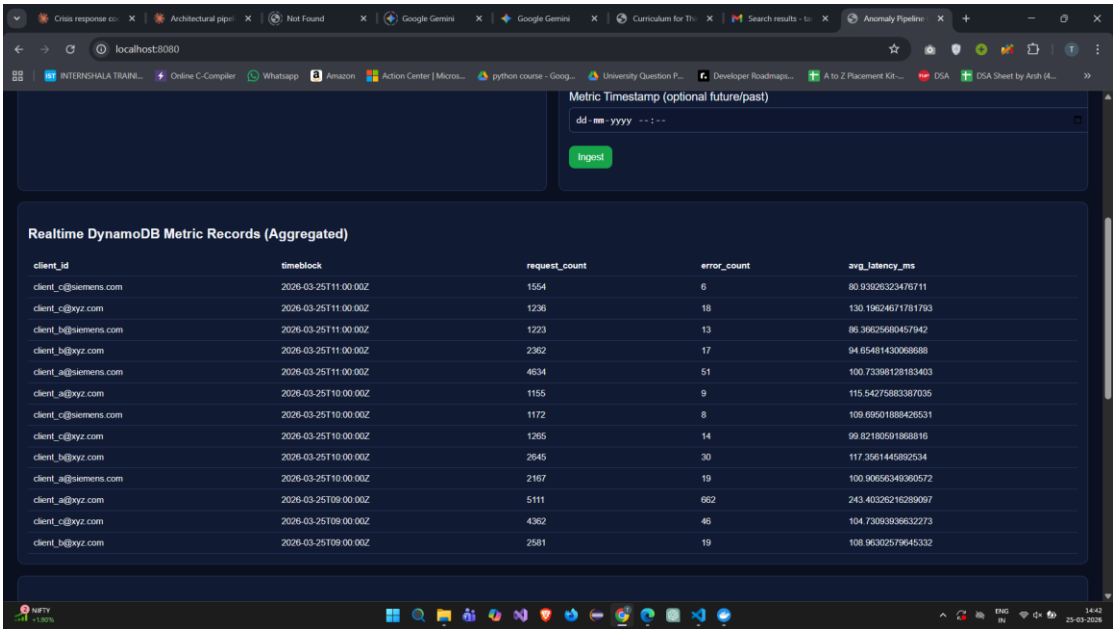

Fig 4 : Realtime table showing the aggregated data & Anomalies being stored in the LocalStack DynamoDB resource

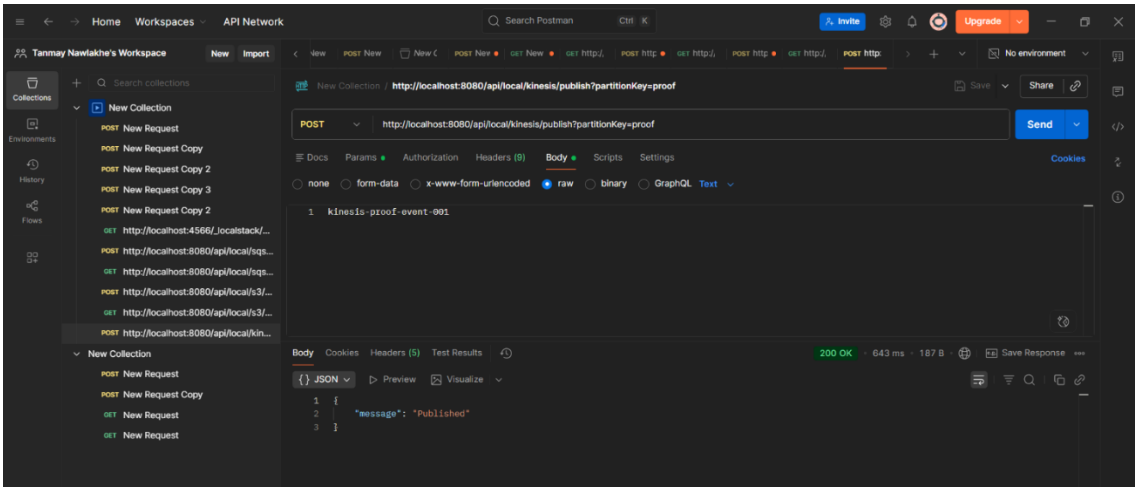

Fig 5 : Localstack Kinesis Service Simlation (Postman HTTP Request)

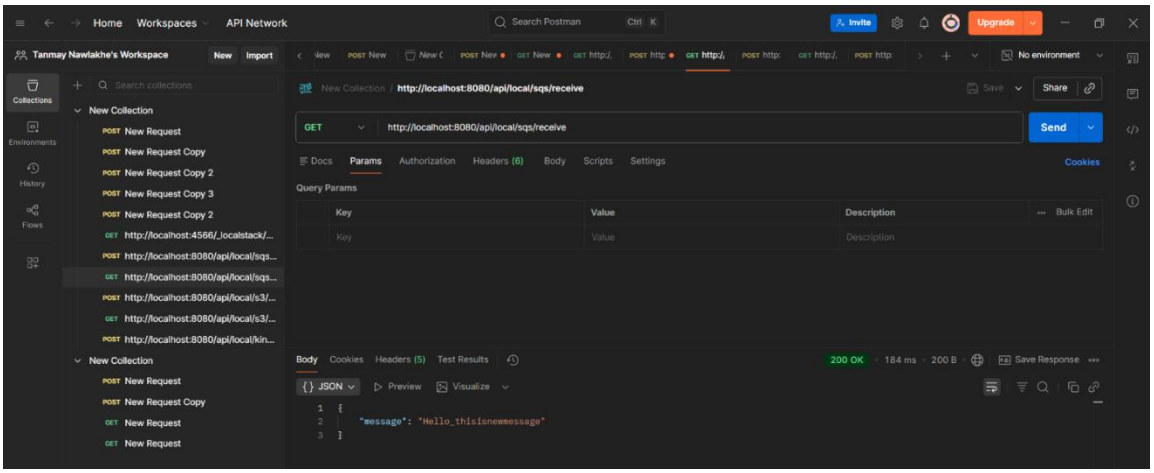

Fig 6 : LocalStack SQS Service Simulation using Postman HTTP Requests

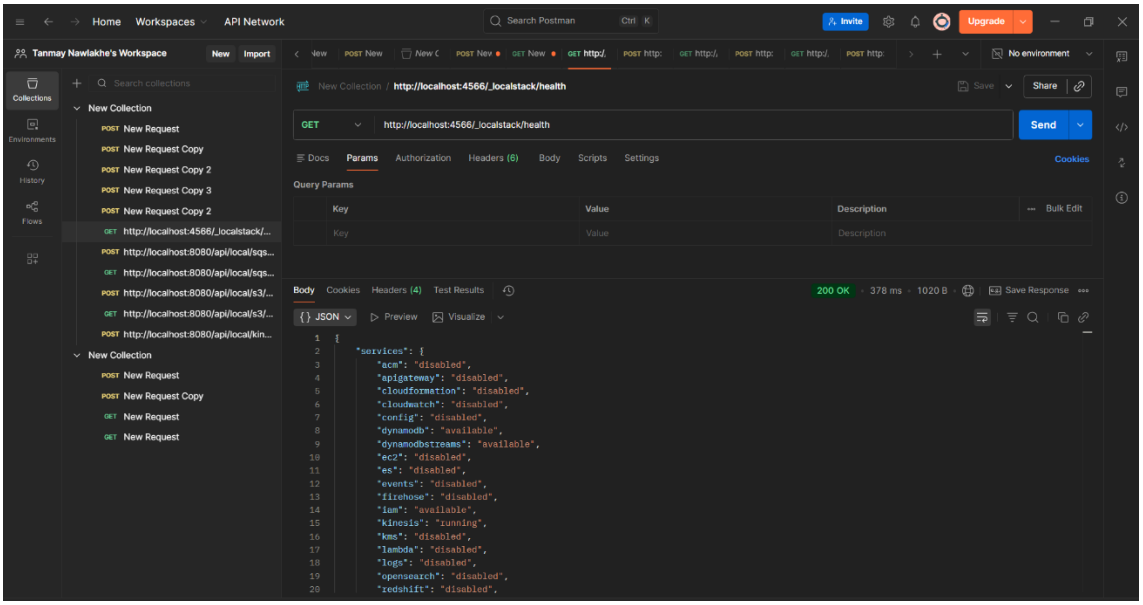

Fig 7 : LocalStack Health Check Endpoint using Postman HTTP Requests